INTERMEDIATE SERIES · GUIDE 2 OF 8

Intermediate MCP

Build production MCP servers, multi-tool flows, auth, SSE transport, testing

01 Transport Modes — stdio vs HTTP/SSE

Transport	How It Works	Best For	Latency
stdio (default)	Host spawns server as subprocess. Communicate via stdin/stdout.	Local tools, filesystem, DB access, CLI integrations	Very low (local process)
HTTP + SSE	Server runs independently on a port. Host connects via HTTP. Server streams events via SSE.	Remote servers, shared team servers, cloud deployment	Network dependent
Streamable HTTP (new)	Replaces SSE in newer MCP spec. Bidirectional streaming over HTTP.	Production remote MCP servers	Low with streaming

```
# HTTP+SSE server (for remote/shared deployment)
from mcp.server.fastmcp import FastMCP

# FastMCP is a higher-level wrapper — much less boilerplate
mcp = FastMCP("my-remote-server")

@mcp.tool()
def search_products(query: str, max_results: int = 5) -> list[dict]:
    """Search the product catalog by keyword."""
    return db.search(query, limit=max_results)

@mcp.resource("catalog://all")
def get_catalog() -> str:
    """Return full product catalog as text."""
    return json.dumps(db.get_all_products())

# Run as HTTP server
if __name__ == "__main__":
    mcp.run(transport="sse", port=8080)
    # Claude Desktop config: {"url": "http://localhost:8080/sse"}
```

02 FastMCP — Rapid Server Development

FastMCP is a high-level Python wrapper that eliminates 80% of the boilerplate. It uses decorators and type hints to auto-generate tool schemas.

```python
# FastMCP — full featured server example
from mcp.server.fastmcp import FastMCP
from pydantic import BaseModel
import httpx

mcp = FastMCP("company-tools")

# Tool with typed inputs — schema auto-generated from type hints
@mcp.tool()
async def get_exchange_rate(from_currency: str, to_currency: str) -> float:
    """Get current exchange rate between two currencies."""
    async with httpx.AsyncClient() as client:
        r = await client.get(
            f"https://api.exchangerate.host/convert",
            params={"from": from_currency, "to": to_currency})
        return r.json()["result"]

# Tool with Pydantic model input
class CustomerQuery(BaseModel):
    customer_id: str
    include_history: bool = False

@mcp.tool()
def get_customer(query: CustomerQuery) -> dict:
    """Fetch customer record from CRM."""
    customer = crm.get(query.customer_id)
    if query.include_history:
        customer["history"] = crm.get_history(query.customer_id)
    return customer

# Resource with URI template
@mcp.resource("customer://{customer_id}")
def get_customer_resource(customer_id: str) -> str:
    return json.dumps(crm.get(customer_id))

# Prompt template
@mcp.prompt()
def analyze_customer(customer_id: str) -> str:
    customer = crm.get(customer_id)
    return f"Analyze this customer record and identify risks: {customer}"
```

03 MCP with Authentication

Production MCP servers need to authenticate callers and manage access to sensitive resources.

```python
# Token-based auth in FastMCP
from mcp.server.fastmcp import FastMCP
from mcp.server.auth import BearerAuthProvider
```

```
def verify_token(token: str) -> dict | None:
    """Verify JWT token, return claims or None if invalid."""
    try:
        return jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
    except jwt.InvalidTokenError:
        return None

mcp = FastMCP("secure-server",
    auth_provider=BearerAuthProvider(verify_token))

@mcp.tool()
def get_sensitive_data(record_id: str, ctx=None) -> dict:
    """Access restricted data — requires valid token."""
    # ctx.auth contains the verified token claims
    user_role = ctx.auth.get("role", "user")
    if user_role != "admin":
        raise PermissionError("Admin access required")
    return db.get_sensitive(record_id)
```

04 Building a RAG MCP Server

A powerful pattern: expose your RAG pipeline as an MCP server. Any MCP-compatible host (Claude Desktop, Cursor, custom apps) can then query your knowledge base.

```
# RAG-as-MCP-server
from mcp.server.fastmcp import FastMCP
from langchain_community.vectorstores import Qdrant
from langchain_openai import OpenAIEmbeddings

mcp = FastMCP("company-knowledge-base")

# Initialize RAG components at startup
embeddings = OpenAIEmbeddings()
vectorstore = Qdrant.from_existing_collection(
    embedding=embeddings,
    url="http://localhost:6333",
    collection_name="company_docs")

@mcp.tool()
def search_knowledge_base(query: str, top_k: int = 5) -> list[dict]:
    """Search company knowledge base for relevant information."""
    docs = vectorstore.similarity_search_with_score(query, k=top_k)
    return [{"content": d.page_content,
             "source": d.metadata.get("source", "unknown"),
             "score": float(score)}
            for d, score in docs]

@mcp.tool()
```

```
def answer_from_knowledge(question: str) -> str:
    """Get a complete RAG-generated answer from company knowledge."""
    docs = vectorstore.similarity_search(question, k=5)
    context = "\n\n".join(d.page_content for d in docs)
    return llm.invoke(f"Context:\n{context}\n\nQ: {question}\nA:")
```

05 Testing MCP Servers

```
# Unit testing MCP tools
import pytest
from mcp import ClientSession
from mcp.client.stdio import stdio_client

@pytest.mark.asyncio
async def test_search_tool():
    """Test that search_products returns correct structure."""
    server_params = StdioServerParameters(
        command="python", args=["my_server.py"])

    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            # List available tools
            tools = await session.list_tools()
            tool_names = [t.name for t in tools.tools]
            assert "search_products" in tool_names

            # Call the tool
            result = await session.call_tool(
                "search_products",
                arguments={"query": "laptop", "max_results": 3})
            assert len(result.content) > 0

# Also use MCP Inspector for interactive testing:
# npx @modelcontextprotocol/inspector python my_server.py
```